

ConcurrentBag Collection Class in C# with Examples

In this article, I am going to discuss the **ConcurrentBag<T> Collection Class in C#** with Examples. Please read our previous article where we discussed [ConcurrentStack Collection Class in C#](#) with Examples. At the end of this article, you will understand the following pointers.

1. [What is ConcurrentBag<T> Class in C#?](#)
2. [Why do we need ConcurrentBag<T> Collection Class in C#?](#)
3. [Generic List<T> Example with Single Thread in C#](#)
4. [Generic List Example using Multiple Threads in C#](#)
5. [Generic List with Locking Mechanism in C#](#)
6. [ConcurrentBag with Multiple Threads in C#](#)
7. [How to Create a ConcurrentBag<T> Collection in C#?](#)
8. [How to Add Elements into a ConcurrentBag<T> Collection in C#?](#)
9. [How to access a ConcurrentBag Collection in C#?](#)
10. [How to Remove Elements from ConcurrentBag<T> Collection in C#?](#)
11. [How to Get the Element from the ConcurrentBag in C#?](#)
12. [How to Copy a ConcurrentBag Collection to an Existing Array in C#?](#)
13. [How to Convert the Con ConcurrentBag to an Array in C#?](#)
14. [ConcurrentBag<T> Collection Class with Complex Types in C#](#)
15. [ConcurrentBag with Producer/Consumer Example in C#](#)

What is ConcurrentBag<T> Class in C#?

The **ConcurrentBag<T>** is a Thread-Safe Collection Class in C#. It was introduced as part of .NET Framework 4.0 and it belongs to `System.Collections.Concurrent` namespace. It allows generic data to be stored in the unordered form. It allows you to store duplicate objects.

The working of the **ConcurrentBag<T>** is very much similar to the working of the **Generic List<T>** Collection Class in C#. The only difference between them is that the **Generic List<T>** Collection is not thread-safe whereas the **ConcurrentBag<T>** is thread-safe. So, we can use the **Generic List** class instead of **ConcurrentBag** with multiple threads, but in that case, as a programmer, it is our responsibility to use explicitly locks to provide thread safety which is not only time-consuming but also error-prone. So, the ideal choice is to use **ConcurrentBag<T>** instead of **Generic List<T>** in a multi-threaded environment, and with **ConcurrentBag<T>**, as a programmer, we don't require to implement any locking mechanism explicitly. The **ConcurrentBag<T>** collection class will take care of the thread safety internally.

Why do we need ConcurrentBag<T> Collection Class in C#?

Let us understand why we need **ConcurrentBag** Collection Class in C# with an Example. So, what we will do here is, first, we will see examples using **Generic List<t>** which stores the elements in the unordered form, then we will see the thread-safety problem with **Generic List** and how we can resolve the thread-safety problem by implementing the locking mechanism explicitly and finally we will see how to use **ConcurrentBag** Collection Class provided by `System.Collections.Concurrent` namespace.

Generic List<T> Example with Single Thread in C#:

In the following example, we created a generic List called MobileOrders to store order information for mobile. Further, if you notice in the below code, the GetOrders method is called from the TestBag method in a regular synchronous way. And from the main method, we simply call the TestBag method.

```
using System;
using System.Collections.Generic;
using System.Threading;
namespace ConcurrentBagDemo
{
    class Program
    {
        static void Main()
        {
            TestBag();
            Console.ReadKey();
        }
        public static void TestBag()
        {
            List<string> MobileOrders = new List<string>();
            GetOrders("Pranaya", MobileOrders);
            GetOrders("Anurag", MobileOrders);
            foreach (var mobileOrder in MobileOrders)
            {
                Console.WriteLine($"Order Placed: {mobileOrder}");
            }
        }
        private static void GetOrders(string custName, List<string> MobileOrders)
        {
            for (int i = 0; i < 3; i++)
            {
                Thread.Sleep(100);
                string order = string.Format($"{custName} Needs {i + 3} Mobiles");
                MobileOrders.Add(order);
            }
        }
    }
}
```

Output:

```
Order Placed: Pranaya Needs 3 Mobiles
Order Placed: Pranaya Needs 4 Mobiles
Order Placed: Pranaya Needs 5 Mobiles
Order Placed: Anurag Needs 3 Mobiles
Order Placed: Anurag Needs 4 Mobiles
Order Placed: Anurag Needs 5 Mobiles
```

As the GetOrders method is called in a synchronous way, the output is also printed similarly i.e. first Pranaya and then Anurag which is what you can see in the above output.

Generic List Example using Multiple-Threads in C#:

Now, let's modify the previous example to make it asynchronous. For that, we have used Task that will call the GetOrders method by using two different threads. And we have done these changes inside the TestBag method as shown in the below code.

```
using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
namespace ConcurrentBagDemo
{
    class Program
    {
        static void Main()
        {
            TestBag();
            Console.ReadKey();
        }
        public static void TestBag()
        {
            List<string> MobileOrders = new List<string>();
            Task t1 = Task.Run(() => GetOrders("Pranaya", MobileOrders));
            Task t2 = Task.Run(() => GetOrders("Anurag", MobileOrders));
            Task.WaitAll(t1, t2); //Wait till both the task completed

            foreach (var mobileOrder in MobileOrders)
            {
                Console.WriteLine($"Order Placed: {mobileOrder}");
            }
        }
        private static void GetOrders(string custName, List<string> MobileOrders)
        {
            for (int i = 0; i < 3; i++)
            {
                Thread.Sleep(100);
                string order = string.Format($"{custName} Needs {i + 3} Mobile");
                MobileOrders.Add(order);
            }
        }
    }
}
```

Now, run the above code multiple times, and each time you might get a different output. That means the output is not consistent as shown in the below image.

```
Order Placed: Pranaya Needs 3 Mobiles
Order Placed: Anurag Needs 3 Mobiles
Order Placed: Anurag Needs 4 Mobiles
Order Placed: Pranaya Needs 4 Mobiles
Order Placed:
Order Placed: Anurag Needs 5 Mobiles
```

Why we are not getting the expected output?

This is because the Add method of Generic List<T> Collection Class in C# is not designed to work with multiple threads parallelly i.e. the Add method of List class is not thread-safe. So, Multi-Threading with Generic List<T> is unpredictable. That means, sometimes it may work but if you try several times, then will get unexpected results.

Generic List with Locking Mechanism in C#:

In the below example, we use the famous lock keyword to the statement i.e. the statement which adds the order to the list collection i.e. the Add method.

```
using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;
namespace ConcurrentBagDemo
{
    class Program
    {
        static object lockObject = new object();
        static void Main()
        {
            TestBag();
            Console.ReadKey();
        }
        public static void TestBag()
        {
            List<string> MobileOrders = new List<string>();
            Task t1 = Task.Run(() => GetOrders("Pranaya", MobileOrders));
            Task t2 = Task.Run(() => GetOrders("Anurag", MobileOrders));
            Task.WaitAll(t1, t2); //Wait till both the task completed

            foreach (var mobileOrder in MobileOrders)
            {
                Console.WriteLine($"Order Placed: {mobileOrder}");
            }
        }
        private static void GetOrders(string custName, List<string> MobileOrders)
        {
            for (int i = 0; i < 3; i++)
            {

```

```

        Thread.Sleep(100);
        string order = string.Format($"{custName} Needs {i + 3} Mobiles");
        lock (lockObject)
        {
            MobileOrders.Add(order);
        }
    }
}

```

Now, run the above code and you will get the output as expected as shown in the below image.

```

Order Placed: Pranaya Needs 3 Mobiles
Order Placed: Anurag Needs 3 Mobiles
Order Placed: Pranaya Needs 4 Mobiles
Order Placed: Anurag Needs 4 Mobiles
Order Placed: Pranaya Needs 5 Mobiles
Order Placed: Anurag Needs 5 Mobiles

```

That is fine. So, getting expected results after putting a lock on the Add method of the Generic List class. But what about if the Add method is called multiple times at multiple locations in our project, would you like to use the lock statement everywhere? If you do so, then it is a time-consuming process as well as error-prone as you might forget to use the lock statement at some places. The solution is to use ConcurrentBag.

ConcurrentBag with Multiple Threads in C#:

The ConcurrentBag provides thread safety automatically in a multi-threaded environment. Let us rewrite the previous example using the ConcurrentBag collection class and see the output and then we will discuss ConcurrentBag collection class in detail. In the following example, we simply replace the List class with ConcurrentBag. And remove the statement used for explicitly locking. Please note the ConcurrentBag class belongs to System.Collections.Concurrent namespace, so include that namespace.

```

using System;
using System.Collections.Concurrent;
using System.Threading;
using System.Threading.Tasks;
namespace ConcurrentBagDemo
{
    class Program
    {
        static object lockObject = new object();
        static void Main()
        {
            TestBag();
            Console.ReadKey();
        }
        public static void TestBag()
        {
            ConcurrentBag<string> MobileOrders = new ConcurrentBag<string>();

```

```

Task t1 = Task.Run(() => GetOrders("Pranaya", MobileOrders));
Task t2 = Task.Run(() => GetOrders("Anurag", MobileOrders));
Task.WaitAll(t1, t2); //Wait till both the task completed
foreach (var mobileOrder in MobileOrders)
{
    Console.WriteLine($"Order Placed: {mobileOrder}");
}
}
private static void GetOrders(string custName, ConcurrentBag<string>
MobileOrders)
{
    for (int i = 0; i < 3; i++)
    {
        Thread.Sleep(100);
        string order = string.Format($"{custName} Needs {i + 3} Mobiles");
        MobileOrders.Add(order);
    }
}
}
}

```

Output:

```

Order Placed: Anurag Needs 5 Mobiles
Order Placed: Anurag Needs 4 Mobiles
Order Placed: Anurag Needs 3 Mobiles
Order Placed: Pranaya Needs 5 Mobiles
Order Placed: Pranaya Needs 4 Mobiles
Order Placed: Pranaya Needs 3 Mobiles

```

Now, I hope you understand the basic need for ConcurrentBag Collection Class in C#. Let us proceed and understand the different methods, properties, and constructors provided by ConcurrentBag collection class in C#.

Methods, Properties, and Constructors of ConcurrentBag Class in C#:

Let us understand the different Methods, Properties, and Constructors of ConcurrentBag Collection Class in C#. If you right-click on the ConcurrentBag class and select go to definition, then you will see the following definition. The ConcurrentBag class belongs to System.Collections.Concurrent namespace and implements IProducerConsumerCollection<T>, IEnumerable<T>, IEnumerable, ICollection, IReadOnlyCollection<T> interfaces.

```

namespace System.Collections.Concurrent
{
    ...public class ConcurrentBag<T> : IProducerConsumerCollection<T>, IEnumerable<T>, IEnumerable, ICollection, IReadOnlyCollection<T>
    {
        ...public ConcurrentBag();
        ...public ConcurrentBag(IEnumerable<T> collection);

        ...public int Count { get; }
        ...public bool IsEmpty { get; }

        ...public void Add(T item);
        ...public void CopyTo(T[] array, int index);
        ...public IEnumerator<T> GetEnumerator();
        ...public T[] ToArray();
        ...public bool TryPeek(out T result);
        ...public bool TryTake(out T result);
    }
}

```

How to Create a ConcurrentBag<T> Collection in C#?

The ConcurrentBag<T> Collection Class in C# provides the following two constructors to create an instance of the ConcurrentBag<T> class.

1. **ConcurrentBag()**: It is used to initialize a new instance of the ConcurrentBag class.
2. **ConcurrentBag(IEnumerable<T> collection)**: It is used to initialize a new instance of the ConcurrentBag class that contains elements copied from the specified collection.

Let's see how to create an instance of ConcurrentBag using the ConcurrentBag() constructor:

Step1:

As the ConcurrentBag<T> class belongs to System.Collections.Concurrent namespace, so first, we need to include System.Collections.Concurrent namespace in our program as follows:

using System.Collections.Concurrent;

Step2:

Next, we need to create an instance of the ConcurrentBag class using the ConcurrentBag() constructor as follows:

ConcurrentBag<type> ConcurrentBag _Name = new ConcurrentBag<type>();

Here, the type can be any built-in data type like int, double, string, etc., or any user-defined data type like Customer, Student, Employee, Product, etc.

How to Add Elements into a ConcurrentBag<T> Collection in C#?

If you want to add elements to a ConcurrentBag collection in C#, then you need to use the following methods of the ConcurrentBag<T> class.

1. **Add(T item)**: This method is used to add an object to the ConcurrentBag. The parameter item specifies the object to be added to the ConcurrentBag. The value can be null for reference types.

For example,

ConcurrentBag<string> concurrentBag = new ConcurrentBag<string>();

The above statement will create a ConcurrentBag to store string elements. So, here we can add only the string values. If we try to add anything other than a string then we will get a compile-time error.

concurrentBag.Add("India");


```
concurrentBag.Add("USA");
concurrentBag.Add(100); //Compile-Time Error
```

We can also add elements into a ConcurrentBag using Collection Initializer as follows:

```
ConcurrentBag<string> concurrentBag = new ConcurrentBag<string>
{
    "India",
    "USA",
    "UK"
};
```

Note: The ConcurrentBag does not provide any AddRange method, so we have to manually call Add method for each item.

How to access a ConcurrentBag Collection in C#?

We can access all the elements of the ConcurrentBag collection in C# using a for each loop as follows.

```
foreach (var item in concurrentBag)
{
    Console.WriteLine(item);
}
```

Example to Understand How to Create a ConcurrentBag and Add Elements in C#:

For a better understanding of how to create a ConcurrentBag, how to add elements, and how to access all the elements from ConcurrentBag in C# using a for-each loop, please have a look at the following example which shows the above three things.

```
using System;
using System.Collections.Concurrent;
namespace ConcurrentBagDemo
{
    class Program
    {
        static object lockObject = new object();
        static void Main()
        {
            //Creating ConcurrentBag collection to store string values
            ConcurrentBag<string> concurrentBag = new ConcurrentBag<string>();
            //Adding Element using Add Method of ConcurrentBag Class
            concurrentBag.Add("India");
            concurrentBag.Add("USA");
            concurrentBag.Add("UK");
            //concurrentBag.Add(100); //Compile-Time Error
            Console.WriteLine("ConcurrentBag Elements");
            foreach (var item in concurrentBag)
            {
                Console.WriteLine(item);
            }
            //Creating a string array and passing the array to ConcurrentBag (
            string[] countriesArray = { "Canada", "NZ", "Japan" };
```

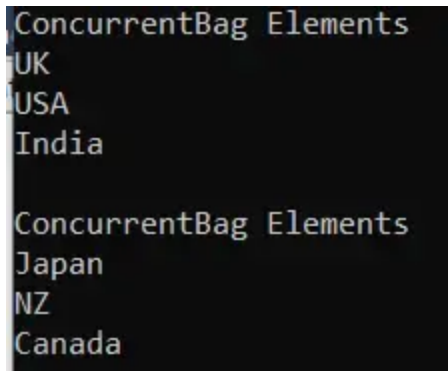


```

        ConcurrentBag<string> concurrentBag2 = new ConcurrentBag<string>
(countriesArray);
        Console.WriteLine("\nConcurrentBag Elements");
        foreach (var item in concurrentBag2)
        {
            Console.WriteLine(item);
        }
        Console.ReadKey();
    }
}

```

Output:



```

ConcurrentBag Elements
UK
USA
India

ConcurrentBag Elements
Japan
NZ
Canada

```

How to Remove Elements from ConcurrentBag<T> Collection in C#?

The ConcurrentBag Collection Class in C# provides the following TryTake method to remove an element from the collection.

1. **TryTake(out T result):** This method attempts to remove and return an object from the ConcurrentBag collection. When this method returns, the result contains the object removed from the ConcurrentBag or the default value of T if the bag is empty. It returns true if an object was removed successfully; otherwise, false.

Let us see an example to understand the TryTake method of ConcurrentBag<T> Collection Class in C#. Please have a look at the following example which shows the use of the TryTake method.

```

using System;
using System.Collections.Concurrent;
namespace ConcurrentBagDemo
{
    class Program
    {
        static object lockObject = new object();
        static void Main()
        {
            //Creating ConcurrentBag collection and Initialize with Collection
            Initializer
            ConcurrentBag<string> concurrentBag = new ConcurrentBag<string>

```

```

        {
            "India",
            "USA",
            "UK",
            "Canada"
        };

        Console.WriteLine("All ConcurrentBag Elements");
        foreach (var item in concurrentBag)
        {
            Console.WriteLine(item);
        }
        //Removing element using TryTake Method
        bool IsRemoved = concurrentBag.TryTake(out string Result);
        Console.WriteLine($"\\nTryTake Return : {IsRemoved}");
        Console.WriteLine($"TryTake Result Value : {Result}");
        Console.WriteLine("\\nConcurrentBag Elements After TryTake Method");
        foreach (var item in concurrentBag)
        {
            Console.WriteLine(item);
        }

        Console.ReadKey();
    }
}

```

Output:

```

All ConcurrentBag Elements
Canada
UK
USA
India

TryTake Return : True
TryTake Result Value : Canada

ConcurrentBag Elements After TryTake Method
UK
USA
India

```

How to Get the Element from the ConcurrentBag in C#?

The ConcurrentBag<T> Collection Class in C# provides the following two methods to get the element from the ConcurrentBag collection.

1. **TryTake(out T result)**: This method attempts to remove and return an object from the ConcurrentBag collection. When this method returns, the result contains the object removed from the ConcurrentBag or the default value of T if the bag is empty. It returns true if an object was removed successfully; otherwise, false.
2. **TryPeek(out T result)**: This method attempt to return an object from the ConcurrentBag without removing it. When this method returns, the parameter result contains an object from the ConcurrentBag or the default value of T if the operation failed. It returns true if an object was returned successfully; otherwise, false.

For a better understanding, please have a look at the below example which shows how to get the element from the ConcurrentBag using the **TryTake(out T result)** and **TryPeek (out T result)** methods of **ConcurrentBag<T>** Collection Class in C#.

```
using System;
using System.Collections.Concurrent;
namespace ConcurrentBagDemo
{
    class Program
    {
        static object lockObject = new object();
        static void Main()
        {
            //Creating ConcurrentBag collection and Initialize with Collection
            Initializer ConcurrentBag<string> concurrentBag = new ConcurrentBag<string>
            {
                "India",
                "USA",
                "UK",
                "Canada",
                "Japan"
            };
            //Printing Elements After TryPeek the Element
            Console.WriteLine($"ConcurrentBag All Elements: Count
{concurrentBag.Count}");
            foreach (var element in concurrentBag)
            {
                Console.WriteLine($"{element} ");
            }

            // Removing and Returning the Element from ConcurrentBag using Try
            bool IsRemoved = concurrentBag.TryTake(out string Result1);
            Console.WriteLine($"\\nTryTake Return : {IsRemoved}");
            Console.WriteLine($"TryTake Result Value : {Result1}");
            //Printing Elements After Removing the Element
            Console.WriteLine($"\\nConcurrentBag Elements After TryTake: Count
{concurrentBag.Count}");
            foreach (var element in concurrentBag)
            {
                Console.WriteLine($"{element} ");
            }
        }
    }
}
```

```

//Returning the Element from ConcurrentBag using TryPeek method
bool IsPeeked = concurrentBag.TryPeek(out string Result2);
Console.WriteLine($"\\nTryPeek Return : {IsPeeked}");
Console.WriteLine($"TryPeek Result Value : {Result2}");
//Printing Elements After TryPeek the Element
Console.WriteLine($"\\nConcurrentBag Elements After TryPeek: Count
{concurrentBag.Count}");
foreach (var element in concurrentBag)
{
    Console.WriteLine($"{element} ");
}

Console.ReadKey();
}
}
}

```

Output:

```

ConcurrentBag All Elements: Count 5
Japan
Canada
UK
USA
India

TryTake Return : True
TryTake Result Value : Japan

ConcurrentBag Elements After TryTake: Count 4
Canada
UK
USA
India

TryPeek Return : True
TryPeek Result Value : Canada

ConcurrentBag Elements After TryPeek: Count 4
Canada
UK
USA
India

```

How to Copy a ConcurrentBag Collection to an Existing Array in C#?

In order to copy a ConcurrentBag Collection to an Existing Array in C#, we need to use the following CopyTo method of the ConcurrentBag Collection Class.

1. **CopyTo(T[] array, int index):** This method is used to copy the ConcurrentBag Elements to an existing one-dimensional Array, starting at the specified array index. Here, the parameter array specifies the one-dimensional array that is the destination of the elements copied from the ConcurrentBag. The Array must have zero-based indexing. The index parameter specifies the zero-based index in the array at which copying begins.

This method works on one-dimensional arrays and does not change the state of the ConcurrentBag. The elements are ordered in the array in the same order as the order of the elements from the beginning of the ConcurrentBag to the end. Let us see an example for a better understanding of the CopyTo(T[] array, int index) method of the ConcurrentBag<T> Collection Class in C#.

```
using System;
using System.Collections.Concurrent;
namespace ConcurrentBagDemo
{
    class Program
    {
        static object lockObject = new object();
        static void Main()
        {
            //Creating ConcurrentBag collection and Initialize with Collection
            Initializer
            ConcurrentBag<string> concurrentBag = new ConcurrentBag<string>
            {
                "India",
                "USA",
                "UK",
                "Canada",
                "Japan"
            };
            //Printing Elements After TryPeek the Element
            Console.WriteLine($"ConcurrentBag All Elements: Count
{concurrentBag.Count}");
            foreach (var element in concurrentBag)
            {
                Console.WriteLine($"{element} ");
            }
            //Copying the concurrentBag to an array
            string[] concurrentBagCopy = new string[5];
            concurrentBag.CopyTo(concurrentBagCopy, 0);
            Console.WriteLine("\nConcurrentBag Copy Array Elements:");
            foreach (var item in concurrentBagCopy)
            {
                Console.WriteLine(item);
            }

            Console.ReadKey();
        }
    }
}
```

Output:

```
ConcurrentBag All Elements: Count 5
Japan
Canada
UK
USA
India

ConcurrentBag Copy Array Elements:
Japan
Canada
UK
USA
India
```

How to Convert the Con ConcurrentBag to an Array in C#?

If you want to convert the ConcurrentBag collection to an array, then you need to use the following ToArray method of the ConcurrentBag<T> collection class in C#.

1. **ToArray()**: This method is used to copy the ConcurrentBag elements to a new array. It returns a new array containing a snapshot of elements copied from the ConcurrentBag.

For a better understanding, please have a look at the below example which shows the use of the ToArray() method of the ConcurrentBag<T> class.

```
using System;
using System.Collections.Concurrent;
namespace ConcurrentBagDemo
{
    class Program
    {
        static object lockObject = new object();
        static void Main()
        {
            //Creating ConcurrentBag collection and Initialize with Collection
            Initializer ConcurrentBag<string> concurrentBag = new ConcurrentBag<string>
            {
                "India",
                "USA",
                "UK",
                "Canada"
            };
            //Printing Elements After TryPeek the Element
            Console.WriteLine($"ConcurrentBag Elements");
            foreach (var element in concurrentBag)
```

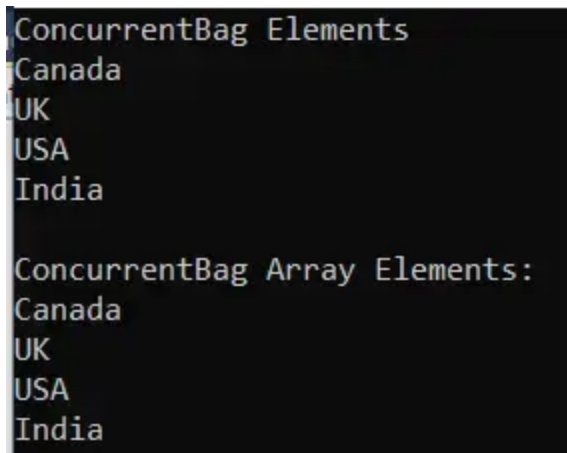
```

        {
            Console.WriteLine($"{element} ");
        }
        //Copying the concurrentBag to an array
        string[] concurrentBagArray = concurrentBag.ToArray();
        Console.WriteLine("\nConcurrentBag Array Elements:");
        foreach (var item in concurrentBagArray)
        {
            Console.WriteLine(item);
        }

        Console.ReadKey();
    }
}

```

Output:



```

ConcurrentBag Elements
Canada
UK
USA
India

ConcurrentBag Array Elements:
Canada
UK
USA
India

```

ConcurrentBag<T> Collection Class with Complex Types in C#

As of now, we have used the ConcurrentBag Collection class with Primitive Data Types such as int, double, etc. Now, let us see how to use the ConcurrentBag Collection with complex types such as Employee, Student, Customer, Product, etc. For a better understanding, please have a look at the below example where we use the ConcurrentBag<T> Collection with the user-defined Student type.

```

using System;
using System.Collections.Concurrent;
namespace ConcurrentBagDemo
{
    class Program
    {
        static void Main()
        {
            //Creating ConcurrentBag to store string values
            ConcurrentBag<Student> concurrentBag = new ConcurrentBag<Student>()
            //Adding Elements to ConcurrentBag using Push Method

```



```

        concurrentBag.Add(new Student() { ID = 101, Name = "Anurag", Branch = "CSE" });
    });
    concurrentBag.Add(new Student() { ID = 102, Name = "Mohanty", Branch = "CSE" });
    });
    concurrentBag.Add(new Student() { ID = 103, Name = "Sambit", Branch = "ETC" });
    });

    //Accessing all the Elements of ConcurrentBag using For Each Loop
    Console.WriteLine($"ConcurrentBag Elements");
    foreach (var item in concurrentBag)
    {
        Console.WriteLine($"ID: {item.ID}, Name: {item.Name}, Branch: {item.Branch}");
    }
    Console.ReadKey();
}
}

public class Student
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }
}
}

```

Output:

```

ConcurrentBag Elements
ID: 103, Name: Sambit, Branch: ETC
ID: 102, Name: Mohanty, Branch: CSE
ID: 101, Name: Anurag, Branch: CSE

```

ConcurrentBag with Producer/Consumer Example in C#:

The ConcurrentBag allows multiple threads to store the objects in the same collection. It is optimized for scenarios where the same thread will act as a producer as well as a consumer. That means the same thread is adding elements as well as retrieving the elements.

For example, let's say we have two threads Thread1 and Thread2. Thread1 added four elements such as 10,20,30,40 to the ConcurrentBag collection. Then Thread2 added three elements as 50,60,70 to the same ConcurrentBag collection. Once both threads added the elements into the collection, Thread1 starts retrieving the data. As Thread1 added 10,20,30,40 elements to the collection, so these elements get preferences over 50,60,70. Once Thread1 retrieves all the four elements which are added by Thread1 then Thread1 goes to retrieve Thread2 inserted elements such as 50,60,70. For a better understanding, please have a look at the below example.

```

using System;
using System.Collections.Concurrent;
using System.Threading;
using System.Threading.Tasks;
namespace ConcurrentBagDemo

```

```

{
    class Program
    {
        static ConcurrentBag<int> concurrentBag = new ConcurrentBag<int>();
        static AutoResetEvent autoEvent1 = new AutoResetEvent(false);
        static void Main(string[] args)
        {
            Task thread1 = Task.Factory.StartNew(() => AddThread1Elements());
            Task thread2 = Task.Factory.StartNew(() => AddThread2Elements());
            Task.WaitAll(thread1, thread2);
            Console.WriteLine("End of the Main Method");
            Console.ReadKey();
        }
        public static void AddThread1Elements()
        {
            int[] array = { 10, 20, 30, 40 };
            for (int i = 0; i < array.Length; i++)
            {
                concurrentBag.Add(array[i]);
            }
            //wait for second thread to add its items
            autoEvent1.WaitOne();
            while (concurrentBag.IsEmpty == false)
            {
                if (concurrentBag.TryTake(out int item))
                {
                    Console.WriteLine($"Thread1 Reads: {item}");
                }
            }
        }
        public static void AddThread2Elements()
        {
            int[] array = { 50, 60, 70 };
            for (int i = 0; i < array.Length; i++)
            {
                concurrentBag.Add(array[i]);
            }
            autoEvent1.Set();
        }
    }
}

```

Output:

```
Thread1 Reads: 40  
Thread1 Reads: 30  
Thread1 Reads: 20  
Thread1 Reads: 10  
Thread1 Reads: 50  
Thread1 Reads: 60  
Thread1 Reads: 70  
End of the Main Method
```

As shown in the above output, when both threads thread1 and thread2 complete adding items, then Thread1 starts retrieving the items. In the bag 50,60,70 is added after the 40,30,20,10 but as Thread1 is accessing item 10,20,30,40 get preferences.

Note: Concurrent Bags are useful for storing objects when order doesn't matter, and unlike sets, bags support duplicates. `ConcurrentBag<T>` is a thread-safe bag implementation, optimized for scenarios where the same thread will be both producing and consuming data stored in the bag. `ConcurrentBag<T>` accepts null as a valid value for reference types.

In the next article, I am going to discuss the [BlockingCollection<T> Class in C#](#) with Examples. Here, in this article, I try to explain the **ConcurrentBag <T> Collection Class in C#** with Examples. I hope this ConcurrentBag Collection Class in C# with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.